1

Introduction to Software Engineering

The amateur software engineer is always in search of magic, some sensational method or tool whose application promises to render software development trivial. It is the mark of the professional software engineer to know that no such panacea exists.

—Grady Booch, in Object-Oriented Analysis and Design

The term software engineering was coined in 1968 as a response to the desolate state of the art of developing quality software on time and within budget. Software developers were not able to set concrete objectives, predict the resources necessary to attain those objectives, and manage the customers' expectations. More often than not, the moon was promised, a lunar rover built, and a pair of square wheels delivered.

The emphasis in software engineering is on both words, *software* and *engineering*. An engineer is able to build a high-quality product using off-the-shelf components and integrating them under time and budget constraints. The engineer is often faced with ill-defined problems, partial solutions, and has to rely on empirical methods to evaluate solutions. Engineers working on application domains such as passenger aircraft design and bridge construction have met successfully similar challenges. Software engineers have not been as successful.

The problem of building and delivering complex software systems on time has been actively investigated and researched. Everything has been blamed, from the customer ("What do you mean I can't get the moon for $50?") to the "soft" in software ("If I could add that one last feature ...") to the youth of this discipline. What is the problem?

Complexity and change.

Useful software systems are complex. To remain useful they need to evolve with the end users' need and the target environment. In this book, we describe object-oriented techniques for conquering complex and changing software systems. In this chapter, we provide a motivation for object-oriented techniques and define the basic concepts used throughout this book.

Introduction Page 4 Thursday, September 2, 1999 12:24 PM

1.1 Introduction: Software engineering failures

Consider the following examples [Neumann, 1995]:

Year 1900 bug

In 1992, Mary from Winona, Minnesota, received an invitation to attend a kindergarten. Mary was 104 at the time.

Leap-year bug

A supermarket was fined $1000 for having meat around 1 day too long, on February 29, 1988. The computer program printing the expiration date on the meat labels did not take into account that 1988 was a leap year.

Interface misuse

On April 10, 1990, in London, an underground train left the station without its driver. The driver had taped the button that started the train, relying on the system that prevented the train from moving when doors were open. The train operator had left his train to close a door which was stuck. When the door was finally shut, the train simply left.

Security

On November 2, 1988, a self-propagating program, subsequently called the Internet Worm, was released into the Internet. The program exploited vulnerabilities in network services, such as the sendmail Unix program, to replicate itself from one computer to another. Unfortunately, the Internet Worm, upon arriving at a machine, consumed all available computing resources and took the infected machine down. An estimated 10% of all Internet nodes were affected. The infection took several days to eradicate.

Late and over budget

In 1995, bugs in the automated luggage system of the new Denver International Airport caused suitcases to be chewed up. The airport opened 16 months late, $3.2 billion over-budget, with a mostly manual luggage system.

On-time delivery

After 18 months of development, a $200 million system was delivered to a health insurance company in Wisconsin in 1984. However, the system did not work correctly: $60 million in overpayments were issued. The system took 3 years to fix.

Unnecessary complexity

The C-17 cargo plane by McDonnell Douglas ran $500 million over budget because of problems with its avionics software. The C-17 included 19 onboard computers, 80 microprocessors, and 6 different programming languages.

Each of the failures described above was due to a software-related problem. In some cases, developers did not anticipate seldom-occurring situations (a person living more than 100 years, leap years impacting expiration dates). In other cases, developers did not anticipate the user actively misusing the system (taping down a button, exploiting sendmail's debugging facilities). In yet other cases, system failures resulted from management failures (late and over-budget delivery, on-time delivery of an incorrect system, unnecessary complexity).

Introduction Page 5 Thursday, September 2, 1999 12:24 PM

Software systems are complex creations: They perform many functions; they are built to achieve many different, and often conflicting, objectives; they comprise many components; many of their components are custom made and complex themselves; many participants, from different disciplines, take part in the development of these components; the development process and the software life cycle often spans many years; finally, complex systems are difficult to understand completely by any single person. Many systems are so hard to understand, even during their development phase, that they are never finished: These are called vaporware.

Software development projects are subject to constant change. Because requirements are complex, they need to be updated when errors are discovered and when the developers have a better understanding of the application. If the project lasts many years, the staff turn-out is high, requiring constant training. The time between technological changes is often shorter than the duration of the project. The widespread assumption of a software project manager that all changes have been dealt with and that the requirements can be frozen lead to an irrelevant system being deployed.

In the next section, we present a high-level view of software engineering. We describe software engineering in the perspective of science, engineering, and knowledge acquisition and formalization. In Section 1.3, we describe in more detail the main terms and concepts we use in this book. In Section 1.4, we provide an overview of the development activities of software engineering. In Section 1.5, we give an overview of the managerial activities of software engineering.

1.2 What is software engineering?

Software engineering is a **modeling** activity. Software engineers deal with complexity through modeling, by focusing at any one time on only the relevant details and ignoring everything else. In the course of development, software engineers build many different models of the system and of the application domain.

Software engineering is a **problem-solving** activity. Models are used to search for an acceptable solution. This search is driven by experimentation. Software engineers do not have infinite resources and are constrained by budget and deadlines. Given the lack of a fundamental theory, they often have to rely empirical methods to evaluate the benefits of different alternatives.

Software engineering is a **knowledge acquisition** activity. In modeling the application and solution domain, software engineers collect data, organize it into information, and formalize it into knowledge. Knowledge acquisition is nonlinear, as a single piece of data can invalidate complete models.

Software engineering is a **rationale-driven** activity. When acquiring knowledge and making decisions about the system or its application domain, software engineers also need to capture the context in which decisions were made and the rationale behind these decisions. Rationale information, represented as a set of issue models, enables software engineers to understand the implication of a proposed change when revisiting a decision.

Introduction Page 6 Thursday, September 2, 1999 12:24 PM

In this section, we describe in more detail software engineering from the perspectives of modeling, problem solving, knowledge acquisition, and rationale. For each of these activities, software engineers have to work under people, time, and budget constraints. In addition, we assume that change can occur at any time.

1.2.1 Modeling

The purpose of sciences is to describe and understand complex systems, such as a system of atoms, a society of human beings, or a solar system. Traditionally, a distinction is made between *natural sciences* and *social sciences* to distinguish between two major types of systems. The purpose of natural sciences is to understand nature and its subsystems. Natural sciences include, for example, biology, chemistry, physics, and paleontology. The purpose of the social sciences is to understand human beings. Social sciences include psychology, and sociology.

There is another type of systems that we call artificial systems. Examples of artificial systems include the space shuttle, airline reservation systems, and stock trading systems. Herbert Simon coined the term *sciences of the artificial* to describe the sciences that deal with artificial systems [Simon, 1970]. Whereas natural and social sciences have been around for centuries, the sciences of the artificial are recent. Computer science, for example, the science dealing with understanding computer systems, is a child of this century.

Many methods that have been successfully applied in natural sciences and humanities can be applied to the sciences of the artificial as well. By looking at the other sciences, we can learn quite a bit. One of the basic methods of science is modeling. A model is an abstract representation of a system that enables us to answer questions about the system. Models are useful when dealing with systems that are too large, too small, too complicated, or too expensive to experience firsthand. Models also allow to visualize and understand systems that either no longer exist or that are only claimed to exist.

Fossil biologists unearth a few bones and teeth preserved from some dinosaur that no one has ever seen. From the bone fragments, they reconstruct a model of the animal, following rules of anatomy. The more bones they find, the clearer their idea of how the pieces fit together and the higher the confidence that their model matches the original dinosaur. If they find a sufficient number of bones, teeth, and claws, they can almost be sure that their model reflects reality accurately, and they can guess the missing parts. Legs, for example, usually come in pairs. If the left leg is found, but the right leg is missing, the fossil biologists have a fairly good idea what the missing leg should look like and where it fits in the model. This is an example of a model of a system that no longer exists.

Today's high-energy physicists are in a similar position to that of a fossil biologist who has found most of the bones. Physicists are building a model of matter and energy and how they fit together at the most basic subatomic level. Their tool is the high-energy particle accelerator. Many years of experiments with particle accelerators have given high-energy physicists enough confidence that their models reflect reality and that the remaining pieces that are not yet found

Introduction Page 7 Thursday, September 2, 1999 12:24 PM

will fit into the so-called standard model. This is an example of a model for a system that is claimed to exist.

Both system modelers, fossil biologists and high-energy physicists, deal with two types of entities: the real-world system, observed in terms of a set of phenomena, and the problem domain model, represented as a set of interdependent concepts. The system in the real world is a dinosaur or subatomic particles. The problem domain model is a description of those aspects of the real-world system that are relevant to the problem under consideration.

Software engineers face similar challenges as fossil biologists and high-energy physicists. First, software engineers need to understand the environment in which the system is to operate. For a train traffic control system, software engineers need to know train signaling procedures. For a stock trading system, software engineers need to know trading rules. Software engineers do not need to become a fully certified train dispatcher or a stock broker; they only need to learn the problem domain concepts that are *relevant* to the system. In other terms, they need to build a model of the problem domain.

Second, software engineers need to understand the systems they could build, to evaluate different solutions and trade-offs. Most systems are too complex to be understood by any one person and are expensive to build. To address these challenges, software engineers describe important aspects of the alternative systems they investigate. In other terms, they need to build a model of the solution domain.

Object-oriented methods combine the problem and solution domain modeling activities into one. The problem domain is first modeled as a set of objects and relationships. This model is then used by the system to represent the real-world concepts it manipulates. A train traffic control system includes train objects representing the trains it monitors. A stock trading system includes transaction objects representing the buying and selling of commodities. Then, solution domain concepts are also modeled as objects. The set of lines used to depict a train or a transaction are objects that are part of the solution domain. The idea of object-oriented methods is that the solution domain model is an extension of the problem domain model. Developing software translates into the activities necessary to identify and describe a system as a set of models that addresses the end user's problem. We describe in more detail modeling and the concepts of objects in Chapter 2, *Modeling with UML*.

1.2.2 Problem solving

Engineering is a problem-solving activity. Engineers search for an appropriate solution, often by trial and error, evaluating alternatives empirically, with limited resources and incomplete knowledge. In its simplest form, the engineering method includes five steps:

1. formulate the problem
2. analyze the problem
3. search for solutions

Introduction Page 8 Thursday, September 2, 1999 12:24 PM

4. decide on the appropriate solution
5. specify the solution

Software engineering is an engineering activity. It is not algorithmic. It requires experimentation, the reuse of pattern solutions, and the incremental evolution of the system toward a solution that is acceptable to the client.

Software development typically includes five development activities: requirements elicitation, analysis, system design, object design, and implementation. During requirements elicitation and analysis, software engineers formulate the problem with the client and build the problem domain model. Requirements elicitation and analysis correspond to steps 1 and 2 of the engineering method. During system design, software engineers analyze the problem, break it down into smaller pieces, and select general strategies for designing the system. During object design, they select detail solutions for each piece and decide on the most appropriate solution. System design and object design result in the solution domain model. System and object design correspond to steps 3 and 4 of the engineering method. During implementation, software engineers realize the system by translating the solution domain model into an executable representation. Implementation corresponds to step 5 of the engineering method. What makes software engineering different from problem solving in other sciences is that change occurs while the problem is being solved.

Software development also includes activities whose purpose is to evaluate the appropriateness of the respective models. During the analysis review, the application domain model is compared with the client's reality, which in turn might change as a result of modeling. During the design review, the solution domain model is evaluated against project goals. During testing, the system is validated against the solution domain model, which might change due to the introduction of new technologies. During project management, managers compare their model of the development process (i.e., the project schedule and budget) against reality (i.e., the delivered work products and expended resources).

1.2.3 Knowledge acquisition

A common mistake that software engineers and managers make is to assume that the acquisition of knowledge needed to develop a system is linear. This mistake is not made by software managers alone. It can be found in other areas as well. In the seventeenth century, a book was published that offered to teach all the German poems by pouring them into the student's head in 6 hours with a funnel.[1] The idea of using a funnel for learning is based on the widespread assumption that our mind is a bucket that is initially empty and can be filled in a linear fashion. Material enters through our senses, accumulates, and is digested. Popper calls this linear acquisition model for knowledge "the bucket theory of the mind." Among the many

1. G. P. Harsdoerfer (1607–1658) "Poetischer Trichter, die teutsche Dicht- und Reimkunst, ohn Behuf der lateinischen Sprache, in 6 Stunden einzugießen," Nuernberg, 1630.

Introduction Page 9 Thursday, September 2, 1999 12:24 PM

other things that are wrong with this theory (described in [Popper, 1992]) is the assumption that knowledge is conceived as consisting of things that can fill a bucket; that is, the fuller the bucket is, the more we know.

Knowledge acquisition is a nonlinear process. The addition of a new piece of information may invalidate all the knowledge we have acquired for the understanding of a system. Even if we had already documented this understanding in documents and code ("The system is 90% coded, we will be done next week"), we must be mentally prepared to start from scratch. This has important implications on the set of activities and their interactions we define to develop the software system. The equivalent of the bucket theory of the mind is the linear waterfall model for software development, in which all steps of the engineering method are accomplished sequentially.

The nonlinearity of the knowledge acquisition process has severe implications on system development. There are several software processes that deal with this problem by avoiding the linear dependencies inherent in the waterfall model. *Risk-based development* attempts to anticipate surprises late in a project by identifying the high-risk components. *Issue-based development* attempts to remove the linearity altogether. Any development activity, be it analysis, system design, object design, implementation, testing, or delivery, can influence any other activity. In issue-based development, all these activities are executed in parallel. The difficulty with nonlinear development models, however, is that they are hard to manage.

1.2.4 Rationale management

When describing the acquisition or evolution of knowledge, we are even less well equipped than when describing the knowledge of an existing system. How does a mathematician derive a proof? Mathematical textbooks are full of proofs but rarely provide hints about the proof derivation. This is because mathematicians do not think of it as being important. Once the axioms and the rules of deduction have been stated, the proof is timeless. It will only have to be revisited when the basic assumptions or deduction rules change. In mathematics, this happens quite rarely. The appearance of non-euclidean geometries 2000 years after Euclid is an example. Non-euclidean geometries are based on the first four axioms of Euclid but assume an alternative axiom in place of the fifth axiom of Euclid (given a line `l` and a point `p` not part of `l`, there is exactly one line `l'` parallel to `l` including point `p`). A noneuclidean geometry was later used by Einstein in his general relativity theory. In astronomy, change is also not a daily event. It took 1500 years to move from Ptolemy's geocentric model of the universe to the Copernican heliocentric model of the universe.

For software engineers, the situation is different. Assumptions that developers make about a system change constantly. Even though the application domain models eventually stabilize once developers acquire an adequate understanding of the problem, the solution domain models are in constant flux. Design and implementation faults discovered during testing and usability problems discovered during user evaluation trigger changes to the solution models. Changes can also be caused by new technology. The availability of a long-life battery and of high-bandwidth

Introduction Page 10 Thursday, September 2, 1999 12:24 PM

wireless communication, for example, can trigger revisions to the concept of a portable terminal. Change introduced by new technology often allows the formulation of new functional or nonfunctional requirements. A typical task to be done by software engineers is to change a currently operational software system to incorporate this new enabling technology. To change the system, it is not enough to understand its current components and behavior. It is also necessary to capture and understand the context in which each design decision was made. This additional knowledge is called the **rationale** of the system.

Capturing and accessing the rationale of a system is nontrivial. First, for every decision made, several alternatives may have been considered, evaluated, and argued. Consequently, rationale represents a much larger amount of information than the solution models do. Second, rationale information is often not explicit. Developers make many decisions based on their experience and their intuition, without explicitly evaluating different alternatives. When asked to explain a decision, developers may have to spend a substantial amount of time recovering its rationale. In order to deal with changing systems, however, software engineers must address the challenges of capturing and accessing rationale.

So far, we have presented a high-level view of software engineering, from the perspectives of modeling, problem solving, knowledge acquisition, and rationale. In the next section, we define the main terms and concepts we use in the book.

1.3 Software engineering concepts

In this section, we describe the main concepts we use throughout the book.[2] A `Project`, whose purpose is to develop a software system, is composed of a number of `Activities` Each `Activity` is in turn composed of a number of `Tasks`. A `Task` consumes `Resources` and produces a `Work Product`. A `Work Product` can be either a `System`, a `Model`, or a `Document`. `Resources` are either `Participants`, `Time`, or `Equipment`. A graphical representation of these concepts is shown in Figure 1-1. This figure is represented in the Unified Modeling Language (UML) notation. We use UML throughout the book to represent models of software and other systems. Intuitively, you should be able to understand this diagram without full knowledge of the UML semantics. Similarly, you can also use UML diagrams when interacting with a client or a user, even though they may not have any knowledge of UML. We describe the semantics of these diagrams in detail in Chapter 2, *Modeling with UML*.

2. As much as possible, we follow the definitions of the IEEE standards on Software Engineering [IEEE, 1997].

Introduction Page 13 Thursday, September 2, 1999 12:24 PM

development of the space shuttle guidance software is to produce a system that is safe (i.e., which does not put human life in danger). The primary goal for a ticket distributor is to produce a system that is highly available (i.e., which functions correctly most of the time).

Goals often conflict; that is, they are difficult to achieve simultaneously. For example, producing a safe system such as a passenger aircraft is expensive. Aircraft manufacturers, however, also need to pay attention to the retail cost of the aircraft, that is, to produce an aircraft that is cheaper than the competition. Safety and low cost are conflicting goals. A substantial amount of complexity in software development comes from ill-defined or conflicting goals.

Requirements are features that the system must have. A **functional requirement** is an area of functionality that the system must support, whereas a **nonfunctional requirement** is a constraint on the operation of the system.

For example, *Provide the user with ticketing information* is a functional requirement. *Providing feedback in less than one second* is a nonfunctional requirement. *Providing a reliable system* is a design goal. *Producing a system at low cost* is a managerial goal. Other constraints include requiring a specific hardware platform for the system or requiring backward compatibility with a legacy system that the client is unwilling to retire.

1.3.6 Notations, methods, and methodologies

A **notation** is a graphical or textual set of rules for representing a model. The Roman alphabet is a notation for representing words. UML (Unified Modeling Language [OMG, 1998]), the notation we use throughout this book, is an object-oriented notation for representing models. Z [Spivey, 1989] is a notation for representing systems based on set theory.

A **method** is a repeatable technique for solving a specific problem. A recipe is a method for cooking a specific dish. A sorting algorithm is a method for ordering elements of a list. Rationale management is a method for justifying change. Configuration management is a method for tracking change.

A **methodology** is a collection of methods for solving a class of problems. A seafood cookbook is a methodology for preparing seafood. The Unified Software Development Process [Jacobson et al., 1999], the Object Modeling Technique (OMT [Rumbaugh et al., 1991]), the Booch methodology [Booch, 1994], and Catalysis [D'Souza & Wills, 1999], are object-oriented methodologies for developing software.

Software development methodologies decompose the process into activities. OMT provides methods for three activities: *Analysis*, which focuses on formalizing the system requirements into an object model, *System Design*, which focuses on strategic decisions, and *Object Design*, which transforms the analysis model into an object model that can be implemented. The OMT methodology assumes that requirements have already been defined and does not provide methods for eliciting requirements. The Unified Software Development Process also includes an *Analysis* activity and treats *System Design* and *Object Design* as a single activity called *Design*. The Unified Process, unlike OMT, includes a *Requirements Capture* activity for eliciting and modeling requirements. Catalysis, while using the same

Introduction Page 14 Thursday, September 2, 1999 12:24 PM

notations as the Unified Process, focuses more on reuse of design and code using patterns and frameworks. All of these methodologies focus on dealing with complex systems.

In this book, we present a methodology for developing complex as well as changing systems. During the course of our teaching and research ([Bruegge, 1992], [Bruegge & Coyne, 1993], [Bruegge & Coyne, 1994], [Coyne et al., 1995]), we have adapted and refined methods from a variety of sources. For activities modeling the application domain, such as requirements elicitation and analysis, we describe methods similar to those of OOSE [Jacobson et al., 1992]. For solution domain modeling activities, such as system design and object design, we describe object-oriented activities similar to those of OMT. For change-related activities, we focus on rationale management, which originated from the design rationale research [Moran & Carroll, 1996], and configuration management, which originated from the maintenance of large systems [Babich, 1986].

1.4 Software engineering development activities

In this section, we give an overview of the technical activities associated with software engineering. Development activities deal with the complexity by constructing models of the problem domain or the system. Development activities include:

- requirements elicitation (Section 1.4.1)
- analysis (Section 1.4.2)
- system design (Section 1.4.3)
- object design (Section 1.4.4)
- implementation (Section 1.4.5)

In Section 1.5, we give an overview of the managerial activities associated with software engineering.

1.4.1 Requirements elicitation

During requirements elicitation, the client and developers define the purpose of the system. The result of this activity is a description of the system in terms of actors and use cases. Actors represent the external entities that interact with the system. Actors include roles such as end users, other computers the system needs to deal with (e.g., a central bank computer, a network), and the environment (e.g., a chemical process). Use cases are general sequences of events that describe all the possible actions between an actor and the system for a given piece of functionality. Figure 1-2 depicts a use case for the ticket distributor example we discussed previously.

Use case name	`PurchaseOneWayTicket`
Participating actor	Initiated by `Traveler`
Entry condition	1. The `Traveler` stands in front of the ticket distributor, which may be located at the station of origin or at another station.
Flow of events	2. The `Traveler` selects the source and destination stations. 3. The `TicketDistributor` displays the price of the ticket. 4. The `Traveler` inserts an amount of money that is at least as much as the price of the ticket. 5. The `TicketDistributor` issues the specified ticket to the `Traveler` and returns any excess change.
Exit condition	6. The `Traveler` holds a valid ticket and any excess change.
Special requirements	If the transaction is not completed after 1 minute of inactivity, the `TicketDistributor` returns all inserted change.

Figure 1-2 An example of use case: `PurchaseOneWayTicket`.

During requirements elicitation, the client and developers also agree on a set of nonfunctional requirements. The following are examples of nonfunctional requirements:

- The ticket distributor should be available to travelers at least 95% of the time.
- The ticket distributor should provide feedback to the traveler (e.g., display the ticket price, return excess change) within 1 second after the transaction has been selected.

We describe requirements elicitation, including use cases and nonfunctional requirements, in detail in Chapter 4, *Requirements Elicitation.*

1.4.2 Analysis

During analysis, developers aim to produce a model of the system that is correct, complete, consistent, unambiguous, realistic, and verifiable. Developers transform the use cases produced during requirements elicitation into an object model that completely describes the system. During this activity, developers discover ambiguities and inconsistencies in the use case model that they resolve with the client. The result of analysis is an object model annotated with attributes, operations, and associations. Figure 1-3 depicts an example of object model for the `TicketDistributor`.

We describe analysis, including object models, in detail in Chapter 5, *Analysis*. We describe in detail the UML notation for representing models in Chapter 2, *Modeling with UML*.

Introduction Page 16 Thursday, September 2, 1999 12:24 PM

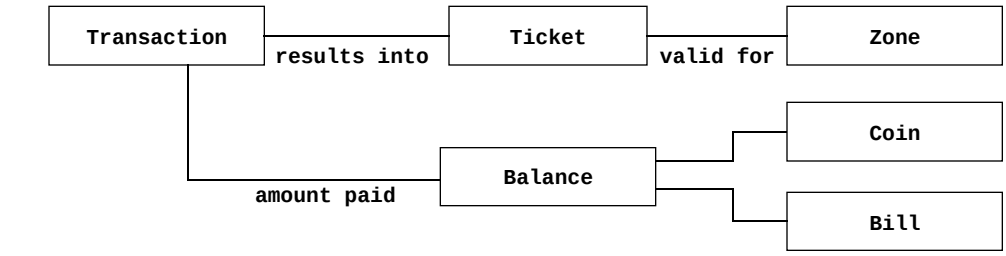

Figure 1-3 An object model for the ticket distributor (UML class diagram). In the `PurchaseOneWayTicket` use case, a `Traveler` initiates a transaction that will result into a `Ticket`. A `Ticket` is valid only for a specified `Zone`. During the `Transaction`, the system tracks the `Balance` due by counting the `Coins` and `Bills` inserted.

1.4.3 System design

During system design, developers define the design goals of the project and decompose the system into smaller subsystems that can be realized by individual teams. Developers also select strategies for building the system, such as the hardware/software platform on which the system will run, the persistent data management strategy, the global control flow, the access control policy, and the handling of boundary conditions. The result of system design is a clear description of each of these strategies, a subsystem decomposition, and a deployment diagram representing the hardware/software mapping of the system. Figure 1-4 depicts an example of system decomposition for the ticket distributor.

We describe system design and its related concepts in detail in Chapter 6, *System Design*.

1.4.4 Object design

During object design, developers define custom objects to bridge the gap between the analysis model and the hardware/software platform defined during system design. This includes precisely describing object and subsystem interfaces, selecting off-the-shelf components, restructuring the object model to attain design goals such as extensibility or understandability, and optimizing the object model for performance. The result of the object design activity is a detailed object model annotated with constraints and precise descriptions for each element. We describe object design and its related concepts in detail in Chapter 7, *Object Design*.

1.4.5 Implementation

During implementation, developers translate the object model into source code. This includes implementing the attributes and methods of each object and integrating all the objects such that they function as a single system. The implementation activity spans the gap between the detailed object design model and a complete set of source code files that can be compiled

Introduction Page 17 Thursday, September 2, 1999 12:24 PM

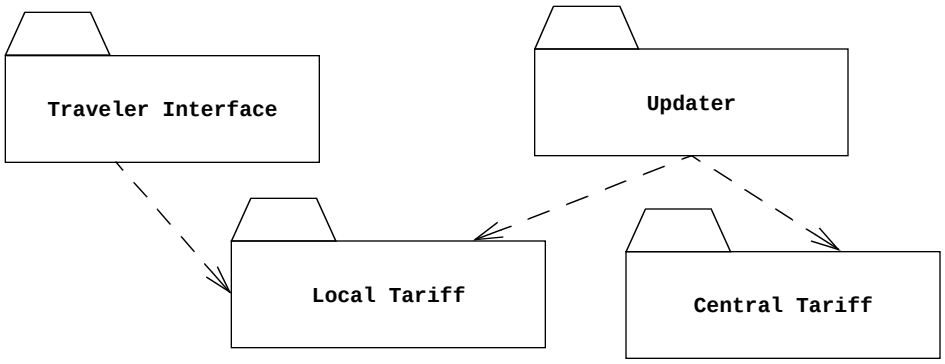

Figure 1-4 A subsystem decomposition for the `TicketDistributor` (UML class diagram, folders represent subsystems, dashed lines represent dependencies). The `TravelerInterface` subsystem is responsible for collecting input from the `Traveler` and providing feedback (e.g., display ticket price, returning change). The `LocalTariff` subsystem computes the price of different tickets based on a local database. The `CentralTariff` subsystem, located on a central computer, maintains a reference copy of the tariff database. An `Updater` subsystem is responsible for updating the local databases at each `TicketDistributor` through a network when ticket prices change.

together. We do not cover any implementation activity in this book, as we assume the reader is already familiar with programming concepts.

1.5 Managing software development

In this section, we briefly describe the activities related with managing a software engineering project. Management activities focus on planning the project, monitoring its status, tracking changes, and coordinating resources such that a high-quality product is delivered on time and within budget. Management activities not only involve managers but also most of the other project participants as well. Management activities include:

- communication (Section 1.5.1)
- rationale management (Section 1.5.2)
- testing (Section 1.5.3)
- software configuration management (Section 1.5.4)
- project management (Section 1.5.5)
- software life cycle modeling activities (Section 1.5.6)

1.5.1 Communication

Communication is the most critical and time-consuming activity in software engineering. Misunderstandings and omissions often lead to faults and delays that are expensive to correct later in the development. Communication includes the exchange of models and documents about

Introduction Page 18 Thursday, September 2, 1999 12:24 PM

the system and its application domain, reporting the status of work products, providing feedback on the quality of work products, raising and negotiating issues, and communicating decisions. Communication is made difficult by the diversity of participants' backgrounds, by their geographic distribution, and by the volume, complexity, and evolution of the information exchanged.

To deal with communication issues, project participants have many tools available. The most effective one is conventions: When participants agree on notations for representing information, on tools for manipulating information, and on procedures for raising and resolving issues, they already have eliminated substantial sources of misunderstanding. Examples of notations include UML diagrams, templates for writing documents and meeting minutes, and identification schemes for naming software components. Examples of tools include Computer Aided Software Engineering (CASE) tools for maintaining models, word processors for generating documents, and interchange formats for publishing information. Examples of procedures include meeting procedures for organizing, conducting, and capturing a meeting, review procedures for reviewing documents and providing feedback, and inspection procedures for detecting defects in models or source code. The selected conventions do not need to be the best available; they only need to be shared and agreed on by everybody. We describe communication issues in detail in Chapter 3, *Project Communication.*

1.5.2 Rationale management

Rationale is the justification of decisions. Given a decision, its rationale includes the problem that it addresses, the alternatives that developers considered, the criteria that developers used to evaluate the alternatives, the debate developers went through to achieve consensus, and the decision. Rationale is the most important information developers need when changing the system. If a criterion changes, developers can reevaluate all decisions that depend on this criterion. If a new alternative becomes available, it can be compared with all the other alternatives that were already evaluated. If a decision is questioned, they can recover its rationale to justify it.

Unfortunately, rationale is also the most complex information developers deal with during development, and thus, the most difficult to update and maintain. To deal with this challenge, developers capture rationale during meetings and on-line discussions, represent rationale with issue models, and access rationale during changes. We describe these issues in detail in Chapter 8, *Rationale Management.*

1.5.3 Testing

During testing, developers find differences between the system and its models by executing the system (or parts of it) with sample input data sets. Although testing is not usually thought of as a management activity, we describe it here because it aims at determining the quality of the system and its related models. During unit testing, developers compare the object design model with each object and subsystem. During integration testing, combinations of

Introduction Page 19 Thursday, September 2, 1999 12:24 PM

subsystems are integrated together and compared with the system design model. During system testing, typical and exception cases are run through the system and compared with the requirements model. The goal of testing is to discover as many faults as possible such that they can be repaired before the delivery of the system. We describe these issues in more detail in Chapter 9, *Testing.*

1.5.4 Software configuration management

Software configuration management is the process that monitors and controls changes in work products. Change pervades software development. Requirements change as the client requests new features and as developers improve their understanding of the application domain. The hardware/software platform on which the system is built changes as new technology becomes available. The system changes as faults are discovered during testing and are repaired. Software configuration management used to be in the realm of maintenance, when improvements are incrementally introduced in the system. In modern development processes, however, changes occur much earlier than maintenance does. As the distinction between development and maintenance is blurred, changes can be dealt with using configuration management at all stages.

Configuration management enables developers to track changes. The system is represented as a number of configuration items that can be independently revised. For each configuration item, its evolution is tracked as a series of versions. Examining and selecting versions enables developers to rollback to a well-defined state of the system when a change fails.

Configuration management also enables developers to control change. After a baseline has been defined, any change needs to be assessed and approved before being implemented. This enables management to ensure that the system is evolving according to project goals and that the number of problems introduced into the system is limited. We describe these issues in detail in Chapter 10, *Software Configuration Management.*

1.5.5 Project management

Project management does not produce any artifact of its own. Instead, project management includes the oversight activities that ensure the delivery of a high-quality system on time and within budget. This includes planning and budgeting the project during negotiations with the client, hiring developers and organizing them into teams, monitoring the status of the project, and intervening when deviations occur. Most project management activities are beyond the scope of this book. We describe, however, the project management activities that are visible to the developers and techniques that make the development/management communication more effective. We describe these issues in detail in Chapter 11, *Project Management.*

1.5.6 Software life cycle

In Section 1.2, we described software engineering as a modeling activity. Developers build models of the application and solution domains to deal with their complexity. By ignoring

Introduction Page 20 Thursday, September 2, 1999 12:24 PM

irrelevant details and focusing only on what is relevant to a specific issue, developers can more effectively resolve issues and answer questions. The process of developing software can also be viewed as a complex system with inputs, outputs, activities, and resources. It is then not surprising, then, that the same modeling techniques applied to software artifacts can be used for modeling software processes. A general model of the software development process is called a software life cycle. We describe software life cycles in the concluding chapter of this book, Chapter 12, *Software Life Cycle.*

1.6 Exercises

1. What is the purpose of modeling?
2. A programming language is a notation for representing algorithms and data structures. List two advantages and two disadvantages of using a programming language as sole notation throughout the development process.
3. Consider a task you are not familiar with, such as designing a zero-emissions car. How would you attack the problem?
4. What is meant by "knowledge acquisition is nonlinear"? Provide a concrete example of knowledge acquisition that illustrates this.
5. Hypothesize a rationale for the following design decisions:
   - "The ticket distributor will be at most one and a half meters tall."
   - "The ticket distributor will include two redundant computer systems."
   - "The ticket distributor will include a touch screen for displaying instructions and inputing commands. The only other control will be a cancel button for aborting a transaction."
6. Specify which of the following statements are functional requirements and which are nonfunctional requirements:
   - "The ticket distributor must enable a traveler to buy weekly passes."
   - "The ticket distributor must be written in Java."
   - "The ticket distributor must be easy to use."
7. Specify which of the following decisions were made during requirements or system design:
   - "The ticket distributor is composed of a user interface subsystem, a subsystem for computing tariff, and a network subsystem managing communication with the central computer."
   - "The ticket distributor will use PowerPC processor chips."
   - "The ticket distributor provides the traveler with an on-line help."
8. What is the difference between a task and an activity?
9. A passenger aircraft is composed of several millions of individual parts and requires thousands of persons to assemble. A four-lane highway bridge is another example of complexity. The first version of Word for Windows, a word processor released by

Introduction Page 21 Thursday, September 2, 1999 12:24 PM

Microsoft in November 1989, required 55 person-years, resulted into 249,000 lines of source code, and was delivered 4 years late. Aircraft and highway bridges are usually delivered on time and below budget, whereas software is often not. Discuss what are, in your opinion, the differences between developing an aircraft, a bridge, and a word processor, which would cause this situation.

References

[Babich, 1986] W. A. Babich, *Software Configuration Management.* Addison-Wesley, Reading, MA, 1986.

[Booch, 1994] G. Booch, *Object-Oriented Analysis and Design with Applications,* 2nd ed. Benjamin/Cummings, Redwood City, CA, 1994.

[Bruegge, 1992] B. Bruegge, "Teaching an industry-oriented software engineering course," *Software Engineering Education,* SEI Conference, Lecture Notes in Computer Sciences, vol. 640, Springer Verlag, San Diego, CA, Oct. 1992, pp. 65–87.

[Bruegge & Coyne, 1993] B. Bruegge & R. Coyne, "Model-based software engineering in larger scale project courses," *IFIP Transactions on Computer Science and Technology,* vol. A-40, pp. 273–287, 1993.

[Bruegge & Coyne, 1994] B. Bruegge & R. Coyne, "Teaching iterative object-oriented development: Lessons and directions," *7th Conference on Software Engineering Education,* Lecture Notes in Computer Science, Vol. 750, Jorge L. Diaz-Herrera (ed.), Springer Verlag, pp. 413–427, Jan. 1994.

[Coyne et al., 1995] R. Coyne, B. Bruegge, A. Dutoit, & D. Rothenberger, "Teaching more comprehensive model-based software engineering: Experience with Objectory's use case approach," *8th Conference on Software Engineering Education,* Lecture Notes in Computer Science, Linda Ibraham (ed.), Springer Verlag, Berlin, pp. 339–374, Apr. 1995.

[D'Souza & Wills, 1999] D. F. D'Souza & A. C. Wills, *Objects, Components, and Frameworks with UML: The Catalysis Approach.* Addison-Wesley, Reading, MA, 1999.

[IEEE, 1997] *IEEE Standards Collection Software Engineering.* IEEE, Piscataway, NJ, 1997.

[Jacobson et al., 1992] I. Jacobson, M. Christerson, P. Jonsson, & G. Overgaard, *Object-Oriented Software Engineering—A Use Case Driven Approach.* Addison-Wesley, Reading, MA, 1992.

[Jacobson et al., 1999] I Jacobson, G. Booch, & J. Rumbaugh, *The Unified Software Development Process.* Addison-Wesley, Reading, MA, 1999.

[Moran & Carroll, 1996] T. P. Moran & J. M. Carroll (eds.), *Design Rationale: Concepts, Techniques, and Use.* Lawrence Erlbaum Associates, Mahwah, NJ, 1996.

[Neumann, 1995] P. G. Neumann, *Computer-Related Risks.* Addison-Wesley, Reading, MA, 1995.

[OMG, 1998] Object Management Group, *OMG Unified Modeling Language Specification.* Framingham, MA, 1998. http://www.omg.org.

[Popper, 1992] K. Popper, *Objective Knowledge: an Evolutionary Approach.* Clarendon, Oxford, 1992.

[Rumbaugh et al., 1991] J. Rumbaugh, M. Blaha, W. Premerlani, F. Eddy, & W. Lorensen, *Object-Oriented Modeling and Design.* Prentice Hall, Englewood Cliffs, NJ, 1991.

[Simon, 1970] H. A. Simon, *The Sciences of the Artificial.* MIT Press, Cambridge, MA, 1970.

[Spivey, 1989] J. M. Spivey, *The Z Notation, A Reference Manual.* Prentice Hall Int., Hertfordshire, U.K. 1989.

2

2.1 Introduction **24**

2.2 An overview of UML **24**

2.2.1 Use case diagrams 25
2.2.2 Class diagrams 25
2.2.3 Sequence diagrams 26
2.2.4 Statechart diagrams 26
2.2.5 Activity diagrams 28

2.3 Modeling concepts **29**

2.3.1 Systems, models, and views 29
2.3.2 Concepts and phenomena 31
2.3.3 Data types, abstract data types, and instances 33
2.3.4 Classes, abstract classes, and objects 33
2.3.5 Event classes, events, and messages 35
2.3.6 Object-oriented modeling 36
2.3.7 Falsification and prototyping 38

2.4 A deeper view into UML **39**

2.4.1 Use case diagrams 39
2.4.2 Class diagrams 45
2.4.3 Sequence diagrams 50
2.4.4 Statechart diagrams 52
2.4.5 Activity diagrams 54
2.4.6 Diagram organization 57
2.4.7 Diagram extensions 59

2.5 Exercises **60**

References **61**